Course: BCA-503 (Fifth Semester)

Software Engineering

Block-I: Introduction to Software Engineering & Project Management

Q1. What is Software Engineering? Explain its importance.

Answer: Introduction:

Software Engineering is the **systematic application of engineering principles** to the development, operation, and maintenance of software. It emphasizes **structured approaches**, planning, documentation, quality assurance, and process management. Unlike ad hoc programming, software engineering ensures that software is **reliable, maintainable, efficient, and cost-effective**.

Explanation:

- Involves **requirements analysis, system design, coding, testing, deployment, and maintenance**.
- Uses **methodologies** such as waterfall, iterative, agile, and spiral.
- Ensures software meets **user expectations** and **quality standards**.

Example:

Developing an **online banking system** requires structured phases: requirements gathering (ATM features), system design (database, transaction modules), coding, testing, and deployment.

Advantages:

1. Produces **high-quality software**.
2. Reduces **time and cost** of development.
3. Enhances **maintainability and reliability**.
4. Supports **team collaboration and large-scale projects**.

Limitations:

1. Requires skilled personnel and experience.
2. Initial planning and documentation can be **time-consuming**.
3. May appear **complex for small/simple projects**.

Q2. Define a Software Project. Explain Size, Quality, and Productivity Factors.

Answer:
Introduction:
A software project is a **temporary endeavor undertaken to create a unique software product**. Projects differ in **size, complexity, and domain**, affecting development strategy and resource planning.

Explanation:

- **Size Factor:** Measured by lines of code, modules, number of features, or functionality points. Large projects need more planning and coordination.
- **Quality Factor:** Ensures reliability, usability, efficiency, maintainability, and security of software. High-quality software reduces **bugs and user dissatisfaction**.
- **Productivity Factor:** Efficiency of development team; measured as output (modules or lines of code) per unit of effort (man-hours).

Example:

- Small Project: Calculator app (2–3 modules).
- Medium Project: Banking software (10 modules).
- Large Project: ERP system for manufacturing (50+ modules).

Importance:

- Helps in **estimating effort, resources, cost, and timelines**.
- Guides **risk assessment** and allocation of responsibilities.

Q3. Explain Phases of Software Development Life Cycle (SDLC).

Answer:
Introduction:
SDLC is a **systematic approach to software development** to improve efficiency, reduce errors, and ensure high-quality outcomes. It defines **steps from requirement analysis to maintenance**.

Phases:

1. **Requirement Analysis:** Collect and document functional and non-functional requirements. Involves stakeholder meetings, questionnaires, and feasibility studies.
2. **System Design:** Transform requirements into **system architecture**. Decide modules, data flow, and system interfaces.
3. **Implementation (Coding):** Developers write code using structured or object-oriented approaches.

4. **Testing:** Detect and fix defects through **unit, integration, system, and acceptance testing**.
5. **Deployment:** Install software in production environment; users start using it.
6. **Maintenance:** Correct defects, enhance features, and update software over time.

Example:
Building an **e-commerce website**:

- Requirement: product catalog, shopping cart, payment gateway.
- Design: database schema, web pages, server architecture.
- Coding: Implement modules in Java/Python.
- Testing: Unit, integration, system testing.
- Deployment: Host website online.
- Maintenance: Add features like discounts, reviews, etc.

Advantages:

- Reduces risks and ensures **organized development**.
- Provides a **structured workflow** for teams.

Limitations:

- Can be **rigid** in traditional models (Waterfall).
- Delays in one phase may affect the entire project.

Q4. Explain Software Project Management (SPM) and Its Key Activities.

Answer:
Introduction:
Software Project Management applies **management principles** to software projects, ensuring they are delivered **on time, within budget, and meet quality standards**.

Key Activities:

1. **Planning:** Define objectives, scope, resources, budget, and schedule.
2. **Scheduling:** Assign tasks to team members with start and end dates.
3. **Monitoring:** Track progress using tools like Gantt charts, PERT charts, and software dashboards.
4. **Controlling:** Correct deviations from plan, manage risks, and reallocate resources if needed.

Example:
A team developing a hospital management system uses a **Gantt chart** to monitor the completion of patient registration, billing, and report generation modules.

Advantages:

- Ensures **timely delivery**.
- Helps **control cost overruns and delays**.
- Improves **team coordination**.

Limitations:

- Requires **skilled managers** and accurate estimation.
- Time-consuming planning stage.

Q5. What is Requirement Specification? Explain Its Importance.

Answer:
Introduction:
Requirement Specification is a **document that clearly defines the software functionality, constraints, and expected behavior**. It ensures that developers, testers, and clients **have a common understanding**.

Explanation:

- **Functional requirements:** Define what the system should do (e.g., "The ATM should dispense cash").
- **Non-functional requirements:** Performance, security, usability, reliability. (e.g., "Response time must be less than 5 seconds").
- Ensures that **developers do not deviate from client expectations**.

Example: ATM software requirements:

- Functional: Cash withdrawal, deposit, balance inquiry.
- Non-functional: Transaction time ≤ 5 seconds, secure PIN authentication.

Importance:

- Prevents **scope creep**.
- Guides **system design and coding**.
- Provides **basis for testing**.

Advantages:

- Reduces errors and miscommunication.
- Helps in **estimating cost and effort**.

Limitations:

- Preparing detailed specification is **time-consuming**.
- Ambiguous or incomplete requirements may lead to defects.

Q6. Explain Software Design Approaches: Function-Oriented vs Object-Oriented Design with examples.

Answer:
Introduction:
Software design converts **requirements into a blueprint** for coding. Two main approaches:

Function-Oriented Design (FOD):

- System divided into **modules or functions**.
- Top-down design approach.
- Example: Banking software: modules `Deposit()`, `Withdraw()`, `BalanceCheck()`.

Object-Oriented Design (OOD):

- Focus on **objects/classes** encapsulating data and methods.
- Supports inheritance, polymorphism, and code reuse.
- Example: `Account` class with attributes `balance`, methods `Deposit()`, `Withdraw()`.

Advantages of OOD:

- Reusable code.
- Easier maintenance.
- Secure encapsulated data.

Limitations of FOD:

- Hard to maintain large systems.
- Less reusable.

Q7. Explain User Interface Design and its considerations.

Answer:
Introduction:
User Interface (UI) design deals with **user interaction with software**. A good UI ensures **usability, intuitiveness, and efficiency**.

Considerations:

- Layout, navigation, input validation, error handling.
- Graphical (GUI) vs Command-line (CLI) interfaces.

Example:
ATM interface: clear buttons, step-by-step instructions, error messages.

Advantages:

- Enhances usability and user satisfaction.
- Reduces training and errors.

Limitations:

- Complex UI increases development time.

Q8. Explain Software Programming and Structured Coding Techniques.

Answer:
Introduction:
Structured coding ensures **modular, readable, and maintainable code**.

Techniques:

- **Top-down design:** High-level design broken into modules.
- **Bottom-up design:** Build basic modules first.
- Use functions, procedures, modular programming.

Example:
Banking software: `CalculateInterest()`, `ProcessTransaction()` functions.

Advantages:

- Easier debugging and testing.
- Better team collaboration.

Limitations:

- Requires careful planning.

Q9. Explain Coding Styles and Standards with examples.

Answer:
Introduction:
Coding style is how code is written; coding standards enforce **uniform practices**.

Example:

```
void deposit(double amount){
    balance += amount;
}
```

- CamelCase, indentation, comments followed.

Advantages:

- Improves readability and maintainability.
- Facilitates collaboration.

Limitations:

- Enforcement slows initial coding.

Q10. What are the advantages of Software Engineering? Explain with examples.

Answer:
Introduction:
Software Engineering provides a **structured and disciplined approach** to software development. It focuses on planning, quality, and process control to ensure **highly reliable and maintainable software**.

Explanation:

- **Quality Improvement:** Reduces bugs and ensures software meets user expectations.
- **Time and Cost Efficiency:** Structured processes reduce development time and avoid rework.
- **Maintainability:** Modular and documented code makes future changes easier.
- **Team Collaboration:** Standard practices and documentation enable large teams to work efficiently.

Example:
Developing an **ERP system** using software engineering ensures modules like inventory, payroll, and sales are **integrated, tested, and maintainable**.

Advantages:

1. High-quality and reliable software.

2. Reduced development time and cost.
3. Easier maintenance and updates.
4. Better coordination in team projects.

Limitations:

- Requires skilled personnel.
- Initial planning and documentation can be time-consuming.

Q11. What are the limitations of Software Engineering? Explain in detail.

Answer:
Introduction:
While software engineering brings many benefits, it also has **limitations** due to its structured approach, cost, and personnel requirements.

Explanation:

- **Complexity:** Large systems require extensive planning and documentation, which can be difficult to manage.
- **Cost and Time:** Initial setup of tools, training, and documentation increases project cost and time.
- **Rigidity:** Traditional models like Waterfall may be too rigid for small or rapidly changing projects.
- **Dependency on Skilled Personnel:** Requires experienced engineers for planning, design, and management.

Example:
Developing a **small mobile app** using full SDLC may be overkill and increase time and cost unnecessarily.

Advantages (if applied correctly):

- Helps large projects succeed despite limitations.

Limitations Summary:

1. High initial cost.
2. Time-consuming.
3. Needs skilled staff.
4. Less flexible for small projects.

Q12. Explain the Phases of Software Project Management (SPM) in detail.

Answer:
Introduction:
Software Project Management applies **management principles to software development**. It ensures that software is delivered **on time, within budget, and meets quality standards**.

Phases:

1. **Planning:** Define project objectives, scope, resources, and timelines.
2. **Scheduling:** Allocate tasks to team members and set milestones.
3. **Monitoring:** Track progress using **Gantt charts, PERT charts, or project management software**.
4. **Controlling:** Adjust plans, allocate additional resources, and handle risks to stay on track.

Example:
Developing a hospital management system:

- Planning: Estimate team size, resources, and milestones.
- Scheduling: Assign deadlines for registration, billing, and reporting modules.
- Monitoring: Weekly progress review meetings.
- Controlling: Adjust resources if the billing module is delayed.

Advantages:

- Ensures timely delivery.
- Optimizes resource utilization.
- Improves coordination among team members.

Limitations:

- Requires accurate estimation and skilled managers.
- Continuous attention and reporting are necessary.

Q13. Explain Planning in Software Project Management with examples and importance.

Answer:
Introduction:
Planning is the first phase of SPM. It involves **defining objectives, estimating resources, and assessing risks** before development begins.

Explanation:

- Define project goals and deliverables.
- Estimate **budget, manpower, and technology requirements**.
- Identify **risks** and create mitigation strategies.

Example:
Planning an ATM software project:

- Determine number of developers needed.
- Estimate time for modules: Cash Withdrawal (2 weeks), Balance Inquiry (1 week).
- Identify risk: Delay in integrating database.

Importance:

- Provides clear direction to the team.
- Reduces risk of delays and cost overruns.
- Ensures resources are used efficiently.

Advantages:

- Helps in **time and cost estimation**.
- Enables risk management.

Limitations:

- Time-consuming.
- Depends on accurate estimates and assumptions.

Q14. Explain Scheduling in Software Project Management with examples.

Answer:
Introduction:
Scheduling is the process of **allocating tasks, setting milestones, and defining timelines** for project completion.

Explanation:

- Assign tasks to developers and teams.
- Determine start and end dates for each module.
- Tools used: **Gantt charts, PERT charts, MS Project**.

Example:
Hospital Management System:

- Patient Registration Module – 2 weeks

- Billing Module – 3 weeks
- Reporting Module – 1 week

Advantages:

- Visual representation of progress.
- Helps in resource allocation.
- Allows early detection of delays.

Limitations:

- Incorrect scheduling can delay the entire project.
- Requires accurate task estimates.

Q15. Explain Monitoring and Controlling in Software Project Management.

Answer:
Introduction:
Monitoring tracks progress, while controlling ensures **corrective measures** are taken to keep the project on schedule and within budget.

Explanation:

- Compare **actual progress vs planned schedule**.
- Identify delays and resource shortages.
- Update schedules and reallocate resources.

Example:
Weekly team meetings review coding and testing milestones. If the billing module is delayed, more developers may be assigned.

Advantages:

- Ensures **on-time and within-budget delivery**.
- Maintains quality standards.
- Helps manage risks effectively.

Limitations:

- Requires continuous monitoring and skilled managers.
- Time-consuming.

Q16. Explain Requirement Specification in detail with types and examples.

Answer:
Introduction:
Requirement specification is a **document that clearly defines what a software system should do** and the constraints under which it operates.

Types:

1. **Functional Requirements:** Define system operations and features.
2. **Non-Functional Requirements:** Performance, reliability, usability, and security.

Example:
ATM software:

- Functional: Cash withdrawal, balance inquiry.
- Non-functional: Response time <5 seconds, secure PIN authentication.

Importance:

- Prevents scope creep.
- Serves as a **basis for design, coding, and testing**.

Advantages:

- Reduces errors and miscommunication.
- Helps in estimating effort and cost.

Limitations:

- Preparing detailed specifications is time-consuming.
- Ambiguous or incomplete requirements may lead to defects.

Q17. Explain Function-Oriented vs Object-Oriented Design Approaches with examples.

Answer:
Introduction:
Software design transforms **requirements into blueprints** for coding. The two approaches are:

Function-Oriented Design (FOD):

- Focus on functions/procedures.
- Top-down decomposition.
- Example: Banking software modules: `Deposit()`, `Withdraw()`.

Object-Oriented Design (OOD):

- Focus on classes and objects.
- Supports encapsulation, inheritance, polymorphism.
- Example: `Account` class with methods `Deposit()` and `Withdraw()`.

Advantages of OOD:

- Reusable code.
- Easier maintenance.
- Secure data through encapsulation.

Limitations of FOD:

- Harder to maintain large systems.

Q18. Explain User Interface Design Considerations with examples.

Answer:
Introduction:
UI design ensures **efficient and intuitive interaction between users and software**.

Considerations:

- Consistency, navigation, feedback, error handling.
- GUI vs CLI interfaces.

Example:
ATM: large buttons, clear error messages, step-by-step instructions.

Advantages:

- Enhances usability and reduces errors.
- Improves user satisfaction.

Limitations:

- Complex UI increases development time.

Q19. Explain Structured Coding Techniques in detail.

Answer:
Introduction:
Structured coding emphasizes **modularity, clarity, and maintainability**.

Techniques:

- Top-down and bottom-up design.
- Use functions/procedures.
- Avoid code duplication.

Example:
Banking software: separate functions for `CalculateInterest()`, `ProcessTransaction()`.

Advantages:

- Easier debugging.
- Team collaboration improves.

Limitations:

- Requires proper planning and discipline.

Q20. Explain Coding Styles and Standards in detail with examples.

Answer:
Introduction:
Coding standards enforce **uniform practices across developers** to ensure maintainable and readable code.

Explanation:

- Naming conventions, indentation, and comments.
- Follows language-specific guidelines (Java, C++, Python).

Example:

```
void deposit(double amount){
    balance += amount;
}
```

- CamelCase naming, proper indentation, and comments used.

Advantages:

- Improves readability and maintainability.
- Facilitates team collaboration.

Limitations:

- Enforcement can slow initial development.

Block-II: Software Verification, Validation & Reliability

Q1. What is Software Verification and Validation (V&V)? Explain its importance.

Answer:
Introduction:
Software Verification and Validation (V&V) ensures that a software product **meets its specified requirements** and **performs as intended**. Verification checks **"Are we building the product right?"**, while Validation checks **"Are we building the right product?"**

Explanation:

- **Verification:** Ensures the software is designed correctly, adheres to specifications, and follows standards.
- **Validation:** Ensures the final software meets user expectations and solves the intended problem.
- Techniques include **reviews, inspections, walkthroughs**, and **testing**.

Example:
Developing an online banking system:

- Verification: Check design documents, coding standards, and module interactions.
- Validation: Test if customers can successfully transfer funds and view balances.

Advantages:

1. Detects defects early.
2. Improves software quality and reliability.
3. Reduces development costs by catching errors before deployment.

Limitations:

1. Time-consuming and requires skilled personnel.
2. Cannot guarantee all errors are detected.

Q2. Explain the theoretical foundations of software testing.

Answer:
Introduction:
Software testing is based on the **principle of systematically evaluating software** to detect defects, ensure correctness, and improve quality.

Explanation:

- Software can have **faults, errors, or failures**. Testing ensures early detection and correction.
- Theoretical foundation includes:
  - **Faults:** Defects in code or design.
  - **Errors:** Human mistakes during coding or design.
  - **Failures:** Occurrence when the software does not behave as intended.
- Testing principles: **exhaustive testing is impossible**, so strategic testing techniques are applied.

Example:
ATM software: fault in withdrawal module may cause incorrect balance display. Testing identifies this before release.

Advantages:

- Improves software reliability.
- Reduces post-deployment failures.

Limitations:

- Cannot catch every defect.
- May require extensive resources for large systems.

Q3. Explain Black Box and White Box Testing approaches with examples.

Answer:
Introduction:
Testing ensures software correctness. Two major approaches are:

Black Box Testing:

- Tests software **without knowing internal implementation**.
- Focuses on **input-output behavior**.
- Example: Entering PIN in ATM and verifying withdrawal amount.

White Box Testing:

- Tests **internal code structure, logic, and flow**.
- Includes **path testing, loop testing, branch coverage**.
- Example: Testing `if-else` conditions in withdrawal module to ensure correct logic.

Advantages:

- Black Box: Simulates user perspective; no need for coding knowledge.
- White Box: Ensures code coverage; detects hidden defects.

Limitations:

- Black Box: Cannot detect internal code issues.
- White Box: Requires source code; time-consuming.

Q4. Explain Integration and System Testing with examples.

Answer:
Introduction:
After unit testing, software modules must be **integrated** and tested as a complete system to ensure they work correctly together.

Integration Testing:

- Tests interactions between modules.
- Types: **Top-down, Bottom-up, and Sandwich approaches**.
- Example: In banking software, integrate `Deposit()` and `BalanceCheck()` modules and test together.

System Testing:

- Tests the **entire software system** against requirements.
- Includes **functional, performance, security, and usability testing**.
- Example: Test end-to-end ATM transaction: insert card → enter PIN → withdraw cash → print receipt.

Advantages:

- Detects interface defects.
- Ensures the software meets requirements.

Limitations:

- Requires complete integration; cannot detect early individual module defects.

Q5. Define Software Reliability. Explain its concepts and factors.

Answer:
Introduction:
Software reliability measures the **ability of software to perform its intended function under specified conditions for a given period of time**.

Concepts:

- **Fault:** Defect in code or logic.
- **Error:** Human mistake causing a fault.
- **Failure:** When software does not work as intended.
- **Repair:** Correcting faults to restore reliability.
- **Availability:** Probability that software is operational at a given time.

Example:
ATM software should reliably process transactions 24/7 without failure.

Advantages:

- Increases user trust.
- Reduces downtime and errors.

Limitations:

- Perfect reliability is almost impossible.
- Requires extensive testing and maintenance.

Q6. Explain Software Faults, Errors, and Failures with examples.

Answer:
Introduction:
Understanding the difference between faults, errors, and failures is crucial for improving software reliability.

Explanation:

- **Faults:** Mistakes in coding or design (e.g., missing a check for negative balance).
- **Errors:** Human mistakes during development (e.g., using wrong variable in calculation).

- **Failures:** Occur when faults or errors manifest in software malfunction (e.g., ATM dispensing wrong cash).

Advantages of Identifying Faults:

- Improves reliability and maintainability.
- Helps prioritize fixes based on severity.

Limitations:

- Some faults are latent and may appear later.

Q7. Explain Repair and Availability concepts in Software Reliability.

Answer:
Introduction:
Software must not only be reliable but also **repairable and available** to users when needed.

Explanation:

- **Repair:** Fixing software defects promptly to restore functionality.
- **Availability:** Probability that the software is operational at any time, calculated as:

Availability=MTBFMTBF+MTTRAvailability = \frac{MTBF}{MTBF + MTTR}Availability=MTBF+MTTRMTBF

Where MTBF = Mean Time Between Failures, MTTR = Mean Time To Repair.

Example:
ATM software: If a fault occurs, quick repair ensures high availability for users.

Advantages:

- Minimizes downtime.
- Improves user confidence.

Limitations:

- Frequent repairs may affect system stability.

Q8. Explain CASE Studies in Software Reliability.

Answer:
Introduction:
CASE (Computer-Aided Software Engineering) studies provide **real-world insight into software reliability, defect management, and testing strategies**.

Example:

1. **Airline Reservation System:** Faults in booking module caused incorrect seat allocation → reliability improved through extensive testing and redundancy.
2. **Banking ATM Systems:** Regular monitoring and patching increased availability and reduced failures.

Advantages:

- Learn from real scenarios.
- Improve testing and reliability practices.

Limitations:

- Case studies may not cover all project types.

Q9. Explain the concept of the Dual of Transportation Model.

Answer:
Introduction:
In operations research, the dual of a transportation problem provides **shadow prices or economic interpretation of constraints**.

Explanation:

- Each supply/demand constraint has an associated dual variable.
- Helps calculate **optimal costs** and assess changes in supply/demand.

Example:
Transportation of goods from factories to warehouses. Dual prices show cost sensitivity if supply changes.

Advantages:

- Helps in cost optimization.
- Supports decision-making.

Limitations:

- Requires linear programming knowledge.

Q10. Explain Economic Interpretation of U and V in Transportation Model.

Answer:
Introduction:
In the transportation model, U and V represent **dual variables associated with supply and demand**.

Explanation:

- U: Associated with origin (supply).
- V: Associated with destination (demand).
- Economic interpretation: Difference between cost and sum of U+V shows optimality or improvement potential.

Example:
If cost of transporting from Factory A to Warehouse B exceeds U+V, adjustments are needed to reduce cost.

Advantages:

- Helps identify potential cost reduction.
- Useful in sensitivity analysis.

Limitations:

- Only applies to linear transportation problems.

Q11. Explain the steps of the MODI Method in Transportation Problems.

Answer:
Introduction:
MODI (Modified Distribution) method is used to **test optimality and improve initial transportation solutions**.

Steps:

1. Obtain initial feasible solution (e.g., Northwest Corner, Least Cost, Vogel's Approximation).

2. Calculate U and V values for occupied cells.
3. Compute opportunity cost for empty cells.
4. If all costs ≥0, solution is optimal; else, adjust allocations.
5. Repeat until optimality achieved.

Advantages:

- Efficient method for finding optimal transportation cost.
- Reduces manual trial and error.

Limitations:

- Complex for large-scale problems.

Q12. Explain Protocols used at each layer of the OSI Reference Model.

Answer:
Introduction:
Protocols define **rules for communication** at each layer of OSI model.

Explanation:

- **Physical Layer:** Ethernet, RS-232.
- **Data Link Layer:** PPP, HDLC.
- **Network Layer:** IP, ICMP.
- **Transport Layer:** TCP, UDP.
- **Session Layer:** RPC, SQL.
- **Presentation Layer:** SSL, JPEG.
- **Application Layer:** HTTP, FTP, Telnet.

Advantages:

- Ensures standardized communication.
- Supports interoperability.

Limitations:

- Protocol overhead may reduce efficiency.

Q13. Describe applications at each OSI layer with examples.

Answer:
Introduction:
OSI layers provide a **framework to design network systems** and applications.

Examples:

- **Physical Layer:** Transmission Media – copper cables, fiber optics, wireless.
- **Data Link Layer:** Repeaters, Hubs, Bridges, Switches.
- **Network Layer:** IP addressing, Subnetting, Routing.
- **Transport Layer:** TCP/UDP for reliable and fast data transfer.
- **Session Layer:** Establishes and maintains communication sessions.
- **Presentation Layer:** Encryption (SSL/TLS), Data formatting (JPEG).
- **Application Layer:** FTP, HTTP, Telnet for end-user services.

Advantages:

- Standardizes network design.
- Ensures interoperability.

Limitations:

- Complexity increases with layers.

Q14. Explain Transmission Media used in networking.

Answer:
Introduction:
Transmission media are the **physical paths or channels** through which data is transmitted from sender to receiver in a network. They are broadly classified into **Guided (wired)** and **Unguided (wireless)** media.

Types:

1. **Guided Media:**
    - Data signals travel along a **physical path**.
    - Examples:
        - **Twisted Pair Cable:** Used in LANs; inexpensive, supports moderate speeds.
        - **Coaxial Cable:** Higher bandwidth; used for cable TV and broadband.
        - **Fiber Optic Cable:** Uses light pulses; very high speed and long distance.
2. **Unguided Media:**
    - Data signals travel **through air or vacuum** without physical medium.

    - Examples: Radio waves, microwaves, infrared, satellite communication.

Advantages:

- Supports data transfer over different distances.
- Choice of media allows cost-effective network design.

Limitations:

- Guided media may require installation effort and cost.
- Unguided media is prone to interference and security issues.

Example:
LAN in an office may use **twisted pair cables**, while Wi-Fi uses **radio waves**.

Q15. Explain the function of Repeaters, Hubs, Bridges, and Switches.

Answer:
Introduction:
Network devices are used to **connect, amplify, and manage data traffic** between nodes in a network.

Explanation:

1. **Repeater:**
    - Amplifies and regenerates signals to extend transmission distance.
    - Works at **Physical Layer**.
2. **Hub:**
    - Connects multiple devices in a LAN; broadcasts incoming data to all ports.
    - Operates at **Physical Layer**.
3. **Bridge:**
    - Connects two LAN segments and filters traffic based on MAC addresses.
    - Operates at **Data Link Layer**.
4. **Switch:**
    - Connects devices intelligently; forwards data only to intended recipient.
    - Operates at **Data Link Layer**; reduces network congestion.

Advantages:

- Extends network coverage (Repeaters).
- Simplifies connections in LAN (Hub/Switch).
- Reduces traffic between segments (Bridge/Switch).

Limitations:

- Hubs cause network congestion due to broadcasting.
- Bridges and Switches are costlier than hubs.

Example:

- Office LAN: Fiber optic cable connects floors; switches on each floor manage device connections.

Q16. Explain IP Addresses and Subnetting.

Answer:
Introduction:
IP addresses are **unique numerical identifiers** assigned to devices in a network. Subnetting divides a large network into **smaller, manageable sub-networks** for efficiency.

Explanation:

- **IPv4:** 32-bit address (e.g., 192.168.1.1).
- **IPv6:** 128-bit address (used for modern internet).
- **Subnetting:**
  - Divides network into smaller parts.
  - Uses **subnet mask** (e.g., 255.255.255.0).
  - Optimizes IP usage and reduces broadcast traffic.

Example:

- Network 192.168.1.0/24 has 256 addresses. Subnetting into /28 creates 16-address subnets.

Advantages:

- Efficient IP allocation.
- Improves network performance and security.

Limitations:

- Requires careful planning.
- Complex for large networks.

Q17. Explain File Transfer Protocol (FTP) and Telnet.

Answer:
Introduction:
FTP and Telnet are **application layer protocols** enabling network services.

FTP (File Transfer Protocol):

- Transfers files between client and server.
- Provides authentication with username and password.
- Modes: Active and Passive.

Telnet:

- Allows **remote login and management** of devices over TCP/IP network.
- Sends commands from client to server.

Example:

- FTP: Uploading website files to a web server.
- Telnet: Logging into a remote router to configure settings.

Advantages:

- FTP: Easy file transfer, supports large files.
- Telnet: Remote access without physical presence.

Limitations:

- FTP: Data and passwords transmitted in plain text (use SFTP for security).
- Telnet: Unencrypted communication; vulnerable to interception.

Q18. Explain Hypertext Transfer Protocol (HTTP) and its role in Internet.

Answer:
Introduction:
HTTP is the **foundation of data communication on the World Wide Web**. It defines rules for **request and response messages** between client (browser) and server.

Explanation:

- HTTP uses **TCP port 80**.
- Works in **stateless mode**; each request is independent.
- HTTPS (secure version) uses **SSL/TLS** for encryption.

Example:

- Accessing www.example.com: Browser sends HTTP GET request → Server responds with HTML page.

Advantages:

- Enables **web browsing and online services**.
- Simple and widely used protocol.

Limitations:

- Stateless nature requires additional mechanisms for session tracking (cookies).
- Vulnerable to security threats without HTTPS.

Q19. Explain Internet Applications at different OSI layers.

Answer:
Introduction:
Various Internet applications operate at different OSI layers to provide end-to-end communication services.

Examples by Layer:

1. **Physical Layer:** Cables, fiber optics, wireless.
2. **Data Link Layer:** Switches, bridges, MAC addressing.
3. **Network Layer:** IP addressing, routing protocols (RIP, OSPF).
4. **Transport Layer:** TCP (reliable), UDP (fast, no reliability).
5. **Session Layer:** Establish and manage sessions (RPC, SQL).
6. **Presentation Layer:** Encryption (SSL/TLS), compression (JPEG, MPEG).
7. **Application Layer:** FTP, HTTP, Telnet, Email.

Advantages:

- Standardized communication.
- Enables interoperability.

Limitations:

- Complexity increases with multiple protocols.
- Configuration errors at one layer can affect others.

Q20. Explain Guided and Unguided Media with examples and applications.

Answer:
Introduction:
Data communication can occur over **Guided (wired) or Unguided (wireless) media**. Proper selection ensures **efficient, reliable, and secure transmission**.

Guided Media:

- Physical path for data: Twisted pair, coaxial, fiber optic.
- Applications: LANs, cable TV networks, high-speed internet.

Unguided Media:

- Wireless transmission: Radio, microwave, infrared, satellite.
- Applications: Wi-Fi, mobile networks, satellite TV.

Advantages:

- Guided: High speed, secure, less interference.
- Unguided: Mobility, flexible deployment.

Limitations:

- Guided: Installation cost and physical constraints.
- Unguided: Interference, limited bandwidth, security concerns.

Example:

- Fiber optic (Guided) for high-speed Internet backbone.
- Wi-Fi (Unguided) for mobile access in offices.